UNIVERSIDADE FEDERAL DE MINAS GERAIS
Instituto de Ciências Exatas — Departamento de Ciência da Computação
Projeto Orientado em Computação I (POC I)

Aprendizado de Máquina para Seleção de Escolhas Gulosas em Algoritmos de Coloração de Grafos

Proposta de Trabalho

Aluno: Lucas Affonso Pires

Tipo de pesquisa: Científica

Orientador: Prof. Marcio Costa Santos

Coorientador: Não há

Belo Horizonte
Agosto de 2026

1. Introdução

O problema de coloração de vértices de um grafo consiste em atribuir cores aos vértices de um grafo $G = (V, E)$ de modo que vértices adjacentes nunca recebam a mesma cor, utilizando o menor número possível de cores — o chamado número cromático do grafo. Trata-se de um problema NP-difícil (KARP, 1972), o que torna inviável, na prática, resolvê-lo de forma exata para instâncias de tamanho realista. Por esse motivo, a abordagem mais utilizada em aplicações reais é o uso de algoritmos gulosos: heurísticas que constroem uma coloração válida percorrendo os vértices em alguma ordem e atribuindo, a cada um, a menor cor ainda não utilizada por seus vizinhos já coloridos.

A qualidade da coloração obtida por um algoritmo guloso depende fortemente da regra usada para decidir, a cada passo, qual vértice colorir em seguida. Heurísticas clássicas como a ordenação por grau decrescente (WELSH; POWELL, 1967), o critério de saturação máxima conhecido como DSATUR (BRÉLAZ, 1979) e a construção por conjuntos independentes RLF (LEIGHTON, 1979) produzem resultados diferentes entre si, e nenhuma delas é sistematicamente superior às demais: comparações empíricas mostram, por exemplo, que o DSATUR supera o algoritmo guloso simples em grafos aleatórios, mas é superado pelo RLF nesse mesmo cenário, enquanto o desempenho relativo se inverte em outras famílias de grafos (LEWIS, 2021). Ou seja, a melhor escolha gulosa em cada passo da construção depende de propriedades estruturais locais do grafo que variam de instância para instância — um problema que se aproxima do que a literatura chama de seleção de algoritmos (algorithm selection) (MUSLIU; SCHWENGERER, 2013).

Esse problema é importante porque a coloração de grafos modela diversas situações práticas de alocação de recursos sob restrição de conflito, como a atribuição de frequências em redes sem fio, a alocação de registradores em compiladores, o escalonamento de exames e a montagem de tabelas de horários. Nesses cenários, algoritmos exatos são computacionalmente inviáveis para instâncias grandes, e uma heurística gulosa mal escolhida pode gerar soluções significativamente piores do que o necessário, com custo direto de recursos (mais frequências, mais registradores, mais salas ou horários). Aprender automaticamente, a partir de dados, a tomar a melhor decisão gulosa em cada passo tem, portanto, potencial de aproximar a qualidade de heurísticas fixas da qualidade de métodos mais sofisticados, sem abrir mão da velocidade que torna os algoritmos gulosos atrativos.

Este trabalho é dividido em dois semestres. O objetivo geral, a ser atingido ao final do POC II, é:

Desenvolver e avaliar um modelo de aprendizado de máquina capaz de aprender, a partir de uma base de dados própria de execuções de algoritmos gulosos de coloração sobre grafos diversos, a tomar — em cada passo da construção gulosa — a decisão que leva a colorações com número de cores igual ou menor do que o obtido pelas heurísticas gulosas clássicas, comparando seu desempenho ao de heurísticas de referência (Welsh-Powell, DSATUR e RLF) em um conjunto representativo de instâncias de benchmark.

Os objetivos específicos do POC I são:

1. Revisar a literatura sobre o problema de coloração de grafos, heurísticas gulosas clássicas e abordagens de aprendizado de máquina aplicadas ao problema.

2. Implementar, em Python, um conjunto de heurísticas gulosas de referência (ordem aleatória, grau decrescente, DSATUR e RLF).

3. Construir uma base de dados diversificada de grafos, combinando modelos aleatórios sintéticos (Erdos–Rényi e Barabási–Albert) e instâncias de benchmark públicas (DIMACS).

4. Executar as heurísticas implementadas sobre essa base, registrando atributos estruturais de cada passo da construção e o número de cores obtido ao final por cada heurística em cada grafo.

5. Realizar uma análise exploratória para identificar relações entre características do grafo/vértice e a heurística que produz o melhor resultado.

6. Implementar uma primeira versão (prova de conceito) de um modelo preditivo que, a partir dos atributos coletados, estime a melhor escolha gulosa em um subconjunto controlado de instâncias.

7. Consolidar a base de dados, o código e os resultados preliminares em um relatório técnico, servindo de ponto de partida para o POC II.

Esses objetivos serão revisitados e detalhados na proposta do POC II, quando os objetivos específicos daquele semestre — voltados à ampliação, ao refinamento e à avaliação sistemática do modelo de aprendizado — serão formalmente definidos com base nos resultados obtidos nesta primeira etapa.

2. Referencial Teórico

2.1 O problema de coloração de grafos

Dado um grafo não direcionado $G = (V, E)$, uma coloração própria de vértices é uma função $c: V \rightarrow \{1, 2, ..., k\}$ tal que $c(u) \neq c(v)$ sempre que $(u, v) \in E$. O número cromático $\chi(G)$ é o menor valor de k para o qual essa coloração existe. Decidir se G admite uma coloração com $k \geq 3$ cores é um problema NP-completo (KARP, 1972), de modo que algoritmos exatos só são práticos para instâncias pequenas ou com estrutura especial.

2.2 Heurísticas gulosas clássicas

O algoritmo guloso sequencial percorre os vértices em uma ordem (fixa ou dinâmica) e atribui a cada um a menor cor ainda não usada por seus vizinhos já coloridos. A qualidade do resultado depende diretamente da ordem escolhida, o que motivou o desenvolvimento de diferentes heurísticas de ordenação:

- **Welsh-Powell:** ordena os vértices por grau decrescente antes de aplicar a construção gulosa (WELSH; POWELL, 1967);

- **DSATUR:** a cada passo, escolhe o vértice ainda não colorido com maior grau de saturação — o número de cores distintas já presentes em sua vizinhança —, desempatando pelo maior grau (BRÉLAZ, 1979);
- **RLF (Recursive Largest First):** constrói a coloração um conjunto independente (uma classe de cor) por vez, selecionando iterativamente vértices que maximizem a cobertura do conjunto independente em formação (LEIGHTON, 1979).

Comparações empíricas sistemáticas (LEWIS, 2021) mostram que essas heurísticas não se dominam mutuamente: o desempenho relativo entre elas varia conforme a densidade e a estrutura do grafo, o que reforça a hipótese central deste trabalho — a de que a melhor escolha gulosa é condicionada a características locais da instância.

2.3 Seleção de algoritmos e aprendizado de heurísticas

Musliu e Schwengerer (2013) tratam explicitamente o problema de escolher, a partir de características do grafo, qual heurística de coloração aplicar — um problema de seleção de algoritmos aplicado à coloração de grafos. Em paralelo, uma linha de pesquisa mais recente propõe aprender diretamente uma política de construção gulosa por meio de redes neurais em grafos (Graph Neural Networks, GNNs). Khalil et al. (2017) propõem um arcabouço geral de aprendizado por reforço sobre embeddings de grafos para aprender heurísticas construtivas gulosas em problemas combinatórios diversos. Lemos et al. (2019) treinam uma GNN de forma supervisionada para estimar a colorabilidade de um grafo com um número fixo de cores. Watkins, Montana e Branke (2023) propõem o método ReLCol, que combina aprendizado por reforço profundo (deep Q-learning) com uma GNN para aprender uma heurística de seleção de vértices competitiva com o DSATUR e o RLF em grafos de benchmark. Langedal e Manne (2024) treinam uma GNN para gerar ordenações de vértices que superam heurísticas gulosas tradicionais tanto em qualidade da coloração quanto em tempo de execução, inclusive em ambientes paralelos.

Esses trabalhos indicam que é possível aprender políticas de decisão competitivas com heurísticas clássicas, mas em geral comparam um número reduzido de heurísticas gulosas fixas contra um único modelo aprendido, sem partir de uma base de dados ampla que relacione, de forma explícita e reutilizável, características locais do grafo ao desempenho de múltiplas heurísticas gulosas simultaneamente. É nesse espaço que este trabalho pretende contribuir, ainda que o aluno reconheça não deter, neste estágio, domínio completo de toda a literatura relacionada — lacuna que será aprofundada ao longo do POC I e revisitada na proposta do POC II.

2.4 Geração e obtenção de instâncias de teste

Para compor uma base de dados diversificada de grafos, este trabalho pretende combinar instâncias de benchmark consolidadas — como as do desafio DIMACS de coloração de grafos (JOHNSON; TRICK, 1996) — com grafos sintéticos gerados por modelos aleatórios amplamente usados na literatura, como o modelo de Erdos–Rényi, em que arestas são geradas independentemente com uma probabilidade fixa (ERDOS; RÉNYI, 1959), e o modelo de Barabási–Albert, que gera grafos com distribuição de grau em lei de potência por meio de ligação preferencial (BARABÁSI;

ALBERT, 1999). Essa combinação permite cobrir tanto instâncias com estrutura controlada quanto instâncias mais próximas de casos reais.

3. Metodologia

O trabalho será conduzido em seis passos principais, descritos a seguir. O ferramental técnico previsto inclui Python como linguagem principal, a biblioteca *networkx* para representação e manipulação de grafos, e bibliotecas usuais de ciência de dados (pandas, scikit-learn) para a etapa de modelagem preditiva.

1. Implementação das heurísticas de referência — implementação em Python dos algoritmos guloso simples, Welsh-Powell, DSATUR e RLF, com testes de corretude sobre instâncias pequenas e conhecidas, garantindo que cada heurística produza sempre colorações válidas.

2. Construção da base de dados de grafos — geração de grafos sintéticos via modelos Erdos–Rényi e Barabási–Albert, variando número de vértices, densidade e parâmetros de geração, complementada por instâncias públicas de benchmark (DIMACS), de modo a cobrir grafos com diferentes tamanhos, densidades e estruturas.

3. Geração dos dados de execução gulosa — execução de cada heurística implementada sobre cada grafo da base, registrando, a cada passo da construção, atributos locais do vértice candidato (grau, grau de saturação, número de vizinhos já coloridos, posição relativa na ordenação) e, ao final, o número total de cores utilizado por cada heurística naquele grafo.

4. Análise exploratória de dados — aplicação de técnicas estatísticas e de visualização, seguindo a lógica das fases de compreensão e preparação de dados da metodologia CRISP-DM, para identificar relações entre características do grafo/vértice e a heurística vencedora, orientando o desenho das variáveis (features) do modelo de aprendizado.

5. Modelagem preditiva inicial — formulação do problema de aprendizado (por exemplo, como classificação: dado um vértice candidato e seu contexto local, prever se ele corresponde à melhor escolha gulosa) e treinamento de um modelo inicial simples sobre um subconjunto controlado de instâncias, como prova de conceito a ser expandida no POC II.

6. Documentação e consolidação — organização do código-fonte, da base de dados gerada e dos resultados preliminares em um relatório técnico, incluindo a documentação necessária para que a base de dados e a implementação sejam reutilizadas e estendidas no POC II.

4. Resultados Esperados

Ao final de um ano de trabalho (POC I e POC II), espera-se obter um modelo de aprendizado de máquina, treinado sobre uma base de dados própria de execuções gulosas de coloração de grafos, capaz de orientar a construção de colorações com qualidade igual ou superior à das heurísticas gulosas clássicas em um conjunto representativo de instâncias, acompanhado de uma análise comparativa de

desempenho — em número de cores utilizado e em tempo de execução — entre o modelo aprendido e as heurísticas de referência (Welsh-Powell, DSATUR e RLF).

Para que esse resultado seja alcançável dentro do prazo previsto, espera-se concluir, ao final do POC I:

- a implementação testada e validada das heurísticas gulosas de referência;
- uma base de dados de grafos diversificada, documentada e reutilizável, com os respectivos resultados de execução de cada heurística;
- uma análise exploratória inicial relacionando características do grafo/vértice à heurística mais eficaz em cada caso;
- uma primeira versão (prova de conceito) do modelo preditivo — mesmo que ainda com desempenho limitado — que sirva de ponto de partida claro e bem documentado para o refinamento a ser realizado no POC II.

5. Etapas e Cronograma

O cronograma a seguir descreve, em resolução semanal, a execução dos passos definidos na Metodologia ao longo do semestre letivo (16 semanas).

Semanas	Atividade
1–2	Revisão bibliográfica e definição detalhada do escopo do trabalho.
3–4	Implementação e validação das heurísticas gulosas de referência (guloso simples, Welsh-Powell, DSATUR, RLF).
5–7	Construção da base de dados de grafos (síntese + benchmark) e execução das heurísticas para geração dos dados.
8–9	Análise exploratória dos dados coletados.
10–12	Definição da formulação de aprendizado e implementação do modelo preditivo inicial (prova de conceito).
13–14	Experimentos preliminares, ajustes e avaliação do modelo de prova de conceito.
15–16	Consolidação da documentação, elaboração do relatório técnico e preparação da proposta de POC II.

6. Referências Bibliográficas

BARABÁSI, A.-L.; ALBERT, R. Emergence of scaling in random networks. *Science*, v. 286, n. 5439, p. 509-512, 1999.

BRÉLAZ, D. New methods to color the vertices of a graph. *Communications of the ACM*, v. 22, n. 4, p. 251-256, 1979.

ERDOS, P.; RÉNYI, A. On random graphs I. *Publicationes Mathematicae Debrecen*, v. 6, p. 290-297, 1959.

JOHNSON, D. S.; TRICK, M. A. (Ed.). *Cliques, Coloring, and Satisfiability*: Second DIMACS Implementation Challenge. Providence: American Mathematical Society, 1996. (DIMACS Series in Discrete Mathematics and Theoretical Computer Science, v. 26).

KARP, R. M. Reducibility among combinatorial problems. In: MILLER, R.; THATCHER, J. (Ed.). *Complexity of Computer Computations*. New York: Plenum Press, 1972. p. 85-103.

KHALIL, E.; DAI, H.; ZHANG, Y.; DILKINA, B.; SONG, L. Learning combinatorial optimization algorithms over graphs. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NEURIPS), 30., 2017. *Anais [...]*. p. 6348-6358.

LANGEDAL, K.; MANNE, F. Graph neural networks as ordering heuristics for parallel graph coloring. *arXiv preprint* arXiv:2408.05054, 2024.

LEIGHTON, F. T. A graph coloring algorithm for large scheduling problems. *Journal of Research of the National Bureau of Standards*, v. 84, n. 6, p. 489-506, 1979.

LEMOS, H.; PRATES, M.; AVELAR, P.; LAMB, L. Graph colouring meets deep learning: effective graph neural network models for combinatorial problems. In: IEEE INTERNATIONAL CONFERENCE ON TOOLS WITH ARTIFICIAL INTELLIGENCE (ICTAI), 31., 2019. *Anais [...]*. p. 879-885.

LEWIS, R. M. R. *A Guide to Graph Colouring*: Algorithms and Applications. 2. ed. Cham: Springer, 2021.

MUSLIU, N.; SCHWENGERER, M. Algorithm selection for the graph coloring problem. In: INTERNATIONAL CONFERENCE ON LEARNING AND INTELLIGENT OPTIMIZATION (LION), 7., 2013. *Anais [...]*. Cham: Springer, 2013. p. 389-403.

WATKINS, G.; MONTANA, G.; BRANKE, J. Generating a graph colouring heuristic with deep Q-learning and graph neural networks. In: INTERNATIONAL CONFERENCE ON LEARNING AND INTELLIGENT OPTIMIZATION (LION), 17., 2023. *Anais [...]*. Cham: Springer, 2023. p. 491-505.

WELSH, D. J. A.; POWELL, M. B. An upper bound for the chromatic number of a graph and its application to timetabling problems. *The Computer Journal*, v. 10, n. 1, p. 85-86, 1967.